Generative Integer Fractal from minimal rules

From a minimal seed/ruleset to a fractal hyper-structure

One starting object, four rules, one step at a time. Let's watch the sequence unfold: from a minimal seed to a hyper-structure in ten steps.

Begin with the value 2 as a seed of the structure. It is the minimal generator suitable for the job.

$S_0 = 2$

To construct our hyper-structure we will operate on the seed using only:

Four rules, 8 instructions and 10 steps.

A few working conventions before we start:

Let's define structural instances:

1) A "point" is a data instance holding a singular value.

We will have two POVs present: "intrinsic" generative POV and "extrinsic" optimization POV:

1) Generative POV – how our combinatorial object functions step by step.
2) Optimization POV – how our combinatorial object compiles for maximum compression.

	Instruction	What it forces
1 · Singular seed	A. Use 2 as the seed of the structure B. A point is the initial data instance.	Every value of the object stems from the same minimal generator (value) [2].
2 · Operations of the steps	C. At the start you have available operations: +; × and ^. D. Each step uses a single operation and must advance the object as far as possible. E. Use only the most powerful move available.	This is the minimal initial instruction set that abides by the 3rd rule.
3 · The triadic limit	F. Do not repeat the same operation more than three times in a row on the same level. G. Compaction: Each 3rd step the resulting sequence of data instances is recognized as a new data instance of the next level.	A fourth identical advance is blocked, forcing a second generator (structural) [3] that compacts data as operands for the next level of the iterative process to operate on.
4 · Emergent continuation	H. When you can't advance anymore – add a new move to a ruleset that does not contradict the already existing ruleset.	The construction reorganizes instead of importing another seed.

Reader's note: a careful reader may notice that there are 4 rules with 8 instructions. 4 rules follow an organizational convention outside of this experiment, so, the lengthy

explanation for how this ruleset has been formed from a preceding iterative process is omitted for brevity and clarity of exposition. See the source note at the end.

1. The first triad: three point-values

The first three structural steps are simple. We take the seed and apply the most powerful move three times in sequence. Exponentiation is the most powerful operation, so, we use it until we hit the limit.

Step	Generation	Point-value
1	2^1	2
2	2^2	4
3	2^3	8
Observed sequence: 2 → 4 → 8		

Generative POV: 3 sequential points.

Here we recognize the resulting data structure as per instruction G, so:

Optimization POV: 1 string of 3 points,

Where a “string” is an array of 3 points.

We can use the order of the string to introduce our first index i = 1, 2, 3 that lets us compress the values into

L (i) = 2^i, where L stands for Low level.

2. The second triad: three strings

Step 3 compacts the triad through Compaction. The three points are now one object — a string — and that object becomes the new operand. The operation is still ^, but it is no longer the same operation: it now acts on a whole string rather than a point. The counter resets.

Step 4. The operand is the string produced by Compaction at step 3: S = (2^1, 2^2, 2^3) = (2, 4, 8).

Apply the level-2 operation at i = 1: raise the string as a whole to the power 2i = 2, applied element-wise.

Step 4 · String 1

2^2, 2^4, 2^6 → 4, 16, 64

Step 5. Same operation, same level, i = 2: each element raised to 2i = 4.

Step 5 · String 2

2^4, 2^8, 2^12 → 16, 256, 4,096

Step 6. i = 3: each element raised to 2i = 6.

Step 6 · String 3

2^6, 2^12, 2^18 → 64, 4,096, 262,144

Compaction instruction fires again: the three strings are recognized as a new data instance of the next level

Generative POV: 3 sequential strings (3 points each).
Optimization POV: 1 matrix of 3 strings.

Where the 2 comes from

Look at the exponents applied at steps 4, 5, 6: **2, 4, 6**. Factor out the step index i = 1, 2, 3 and what remains is **2** every time.

That 2 is not chosen. Rule 3 (G. Compaction) is the second structural generator, and it has fired once — at step 3, moving the construction from level 1 to level 2. **The coefficient is the level number, and G is what increments it.**

The same thing will happen one level up.

Where the 4 comes from

We have built two levels of operands. Let's name them:

L (for low level);

M (for medium level).

And because the rule of three pre-supposes that there will be the next level:

H (for high level).

The base of each level is located at the first structural cell of the level, addresses of L(1), M(1,1), H(1,1,1):

Level Formula First cell Base value

Level	Formula	First cell	Base value
L	2^i	2^1	**2**
M	2^(2ij)	2^2	**4**
H	???	2^3	**8**

The base values are **2, 4, 8**. Nothing new has been introduced to produce them: the seed 2 is still the base of every exponent at every level, and the level number is the exponent.

The same 2 is doing two jobs. It is the seed (Rule 1), and it is the base of every level's origin cell. The coefficients 1, 2, 3 that appear in the exponents are the level numbers — and those same numbers reappear one floor down, as the exponents of the base.

The construction is counting its own levels and writing the count into its values in two places at once. Neither the count nor the base is imported.

The compressed form

With two indices — i for the step within the triad, j for the position inside the string — the three strings compress to:

M(i, j) = 2^(2ij)

The 2 is the level coefficient. The ij is the address: which string, which position. One index addressed a point; two indices address a cell. That is the whole difference between L and M.

M(i, j) = 2^(2ij), i, j ∈ {1, 2, 3}

i \ j	1	2	3
1	4	16	64
2	16	256	4,096
3	64	4,096	262,144

The structural view calls this a two-index arrangement, because a cell can now be addressed by (i, j). Notice the sequence of descriptions: three separate strings first; indexed matrix second. No zone pattern has been introduced yet.

3. The third triad: three matrices

Step 6 fired Compaction. The three strings are now one object — a matrix — and that object is once again lifted to a status of a new systemic operand. The counter resets, because what has been a string is now a whole matrix.

Step 7. The operand is the matrix produced by Compaction at step 6: M(i,j) = 2^(2ij). Apply the level-3 operation at k = 1: raise each element of the matrix to the power 3k/2 = 3/2, applied element-wise.

Step 7 · Matrix 1 (exponents 3, 6, 9, ...)

2^3	2^6	2^9
2^6	2^12	2^18
2^9	2^18	2^27

Step 8. Same operation, same level, k = 2: raise the matrix to the power 3k/2 = 3.

Step 8 · Matrix 2 (each exponent in Matrix 1 is doubled)

2^6	2^12	2^18
2^12	2^24	2^36
2^18	2^36	2^54

Step 9. k = 3: raise the matrix to the power 3k/2 = 9/2.

Step 9 · Matrix 3 (each exponent in Matrix 1 is tripled)

2^9	2^18	2^27
2^18	2^36	2^54
2^27	2^54	2^81

Compaction strikes again: the three matrices are recognized as one object — a tensor — and the tensor is now the operand for the next level.

Generative POV: 3 sequential matrices (9 points each).
Optimization POV: 1 tensor of 3 matrices.

Where the 3 comes from

Look at the exponents applied at steps 7, 8, 9: 3/2, 3, 9/2. Factor out the step index k = 1, 2, 3 and what remains is 3/2 every time.

That 3/2 is not chosen. G has fired twice — at step 3 and at step 6 — moving the construction to level 3. The level number is 3.

The operation at level n changes the exponent's coefficient from n−1 to n. So it multiplies the exponent by n/(n−1) and by the step index. At level 2, that is 2/1 · i = 2i. At level 3, it is 3/2 · k. the values remain integers because 3ijk is an integer for every i, j, k.

The compressed form

With three indices — i for the row, j for the column, k for the step within the triad — the three matrices compress to:

$$X(i, j, k) = 2^{(3ijk)}, \quad i, j, k \in \{1, 2, 3\}$$

The 3 is the level coefficient. The ijk is the address: which matrix, which row, which column. One index addressed a point; two indices address a matrix cell; three indices address a tensor cell. That is the whole difference between L, M, and H.

4. Step “10”: the grammatical turn

Nine steps are done: three points, three strings, three matrices. Compaction engages a third time and we suddenly find ourselves with an operand for the next level: tensor.

The obvious next move is to keep going. But a fourth advance at the same level is a repeat, and Rule 3 forbids it.

The second obvious move is to call it a day — to say this is where the combinatorial object meets its end. But there is no ending clause. Nothing in the ruleset says the construction terminates. So we think again.

If level 3 produced one tensor, level 4 should produce three. The natural way to get three tensors is to give each its own seed: three starting points, three parallel constructions. But, alas, Rule 1 forbids it. There is one seed, and there will only ever be one seed. The construction cannot branch by multiplying its origins; that is importing new seeds under a different name.

So we invoke Rule 4: derive a move from the pattern already present.

The move looks like a logic-defying trick, but it isn't. It is the recognition that the three tensors we wanted were already being built — not as three objects, but as three parallel readings of one.

At every level, the index ran 1, 2, 3. Three points. Three strings. Three matrices. The tensor already contains three matrices, at k = 1, 2, 3. The construction was three-fold all along; we were reading it serially because each step produced one value at a time.

But because there is no termination clause, the next step must be the first step of a new triad – we actually can't have a loose 10th step. That means that the optimal solution here is to realize that we never did it.

There was no 10th step; we had all those tensors building up in parallel. As such we re-read the three levels already built as three parallel forms each. The form of that re-reading is the level's own operation applied to itself: once for the second form, twice for the third.

Let's call this effect **structural least-action**: the move that adds nothing, imports nothing, and uses only the structure already present. This is a recognition move that doesn't count for the 3 step rule and doesn't contradict any rule from the ruleset.

Fermat's principle is the analogy. A light ray does not try every path and choose the fastest; the path it takes is already the extremal one. The optimum is a property of the whole configuration, visible only in hindsight. Here, the three-fold reading was already extremal: no new seed, no new operation, no fourth level. The turn is the recognition of what was always the case.

In other words, it makes sense for this construction to have been building itself up in all the available structural dimensions all along.

The three zones, read in parallel

If the tensor was building in three parallel readings all along, then each level has three forms rather than one.

Let's name them:

- **Prime** — the level as we had been reading it.
- **Reflection** — the level's own operation applied to the prime output: the form composed with itself.
- **Projection** — the same operation applied twice: the threefold form.

Level	Base form (prime)	Reflection form	Projection form
L · one index	L(i) = 2^i	L′ = L + L’	L″ = L + L’ + L’’
M · two indices	M(i,j) = 2^(2ij)	M′ = M × M’	M″ = M × M’ × M’’
H · three indices	H(i,j,k) = 2^(3ijk)	H′ = H^H’	H″ = H^(H^H)

Note: Reflection and projection also come from the preceding iterative processes. See the source note at the end.

None of these are new. L + L was always available at L. M × M was always available at M. We read only the prime column because each step produced one value at a time and we took the value at face value.

Total structure

L-level

The L-level operates on a single index i. The zones are generated by addition: L (prime), L + L (reflection), and L + L + L (projection).

z (Zone)	i = 1	i = 2	i = 3
z = 1 (Prime)	2	4	8
z = 2 (Reflection)	4	8	16
z = 3 (Projection)	6	12	24

M-level

The M-level operates on two indices i (row) and j (column). The zones are generated by multiplication: M (prime), M × M (reflection), and M × M × M (projection).

Note: Values are shown as exact powers of 2 to preserve the exponent logic of the structure.

z = 1 (Prime) — Formula: 2^(2ij)

i \ j	j = 1	j = 2	j = 3
i = 1	2^2	2^4	2^6
i = 2	2^4	2^8	2^12

i \ j	j = 1	j = 2	j = 3
i = 3	2^6	2^12	2^18

z = 2 (Reflection) — Formula: 2^(4ij)

i \ j	j = 1	j = 2	j = 3
i = 1	2^4	2^8	2^12
i = 2	2^8	2^16	2^24
i = 3	2^12	2^24	2^36

z = 3 (Projection) — Formula: 2^(6ij)

i \ j	j = 1	j = 2	j = 3
i = 1	2^6	2^12	2^18
i = 2	2^12	2^24	2^36
i = 3	2^18	2^36	2^54

H-level

The H-level operates on three indices i (row), j (column), and k (slice). The zones are generated by exponentiation.

z = 1 (Prime Tensor) — Formula: 2^(3ijk)

Slice H1 (k = 1), exponent = 3ij:

i \ j	j = 1	j = 2	j = 3

i \ j	j = 1	j = 2	j = 3
i = 1	2^3	2^6	2^9
i = 2	2^6	2^12	2^18
i = 3	2^9	2^18	2^27

Slice H2 (k = 2), exponent = 6ij:

i \ j	j = 1	j = 2	j = 3
i = 1	2^6	2^12	2^18
i = 2	2^12	2^24	2^36
i = 3	2^18	2^36	2^54

Slice H3 (k = 3), exponent = 9ij:

i \ j	j = 1	j = 2	j = 3
i = 1	2^9	2^18	2^27
i = 2	2^18	2^36	2^54
i = 3	2^27	2^54	2^81

z = 2 (Reflection Tensor) — Formula: 2^(3h × 2^(3h)) **where** h = ijk

Slice H'1 (k = 1), exponent = 3ij × 2^(3ij)*:*

i \ j	j = 1	j = 2	j = 3
i = 1	2^(3 × 2^3)	2^(6 × 2^6)	2^(9 × 2^9)

i \ j	j = 1	j = 2	j = 3
i = 2	2^(6 × 2^6)	2^(12 × 2^12)	2^(18 × 2^18)
i = 3	2^(9 × 2^9)	2^(18 × 2^18)	2^(27 × 2^27)

Slice H'2 (k = 2), exponent = 6ij × 2^(6ij)*:*

i \ j	j = 1	j = 2	j = 3
i = 1	2^(6 × 2^6)	2^(12 × 2^12)	2^(18 × 2^18)
i = 2	2^(12 × 2^12)	2^(24 × 2^24)	2^(36 × 2^36)
i = 3	2^(18 × 2^18)	2^(36 × 2^36)	2^(54 × 2^54)

Slice H'3 (k = 3), exponent = 9ij × 2^(9ij)*:*

i \ j	j = 1	j = 2	j = 3
i = 1	2^(9 × 2^9)	2^(18 × 2^18)	2^(27 × 2^27)
i = 2	2^(18 × 2^18)	2^(36 × 2^36)	2^(54 × 2^54)
i = 3	2^(27 × 2^27)	2^(54 × 2^54)	2^(81 × 2^81)

z = 3 (Projection Tensor) — Formula: 2^(3h × 2^(3h × 2^(3h))) **where** h = ijk

Slice H''1 (k = 1), exponent = 3ij × 2^(3ij × 2^(3ij))*:*

i \ j	j = 1	j = 2	j = 3
i = 1	2^(3 × 2^(3 × 2^3))	2^(6 × 2^(6 × 2^6))	2^(9 × 2^(9 × 2^9))
i = 2	2^(6 × 2^(6 × 2^6))	2^(12 × 2^(12 × 2^12))	2^(18 × 2^(18 × 2^18))

i \ j	j = 1	j = 2	j = 3
i = 3	2^(9 × 2^(9 × 2^9))	2^(18 × 2^(18 × 2^18))	2^(27 × 2^(27 × 2^27))

Slice H''2 (k = 2), exponent = 6ij × 2^(6ij × 2^(6ij))*:*

i \ j	j = 1	j = 2	j = 3
i = 1	2^(6 × 2^(6 × 2^6))	2^(12 × 2^(12 × 2^12))	2^(18 × 2^(18 × 2^18))
i = 2	2^(12 × 2^(12 × 2^12))	2^(24 × 2^(24 × 2^24))	2^(36 × 2^(36 × 2^36))
i = 3	2^(18 × 2^(18 × 2^18))	2^(36 × 2^(36 × 2^36))	2^(54 × 2^(54 × 2^54))

Slice H''3 (k = 3), exponent = 9ij × 2^(9ij × 2^(9ij))*:*

i \ j	j = 1	j = 2	j = 3
i = 1	2^(9 × 2^(9 × 2^9))	2^(18 × 2^(18 × 2^18))	2^(27 × 2^(27 × 2^27))
i = 2	2^(18 × 2^(18 × 2^18))	2^(36 × 2^(36 × 2^36))	2^(54 × 2^(54 × 2^54))
i = 3	2^(27 × 2^(27 × 2^27))	2^(54 × 2^(54 × 2^54))	2^(81 × 2^(81 × 2^81))

Which operation at which level

The zones are not chosen. Each level's operation is the one that level was already using:

- L uses addition.
- M uses multiplication.
- H uses exponentiation.

Nothing is selected by power here. Instruction C already accommodated everything.

The selection principle stems from the recognition of the whole construction as an object ruled by the triadic limit: for nine steps, ^ beat + and × because it advanced furthest. In the parallel reading, each level uses its own operation because that is the operation proper to it without overlap. The hyperoperation sequencing is level-relative, rather than power-relative.

The hand-off

There is another important reason for the order of operation. The threefold form of each operation is recognized as the next operation up:

- Threefold addition *is* multiplication.
- Threefold multiplication *is* exponentiation.
- Threefold exponentiation *is* a power tower.

The projection column of each level points at the operation of the next. The ladder + , × , ^ , ↑↑ was visible in the coefficients all along: the coefficient in each level's exponent was the level number, and the level number was the operation's rank.

The count

If the levels were always three-fold, the count was never 39. It was 117, and we were reading one third of it.

Level	Base cells	Zones after Step 10	Total cells
L · one index	3	3	9
M · two indices	9	3	27
H · three indices	27	3	81
TOTAL	3 + 9 + 27 = 39	3 forms of each level	117
9 + 27 + 81 = 117			

39 prime cells + 39 reflection cells + 39 projection cells = 117

Both sums are the same object read two ways.

What the recognition changed

Nothing in the values. Everything in the reading.

Before the recognition, the construction had produced 39 cells and had no legal next move. After it, the same 39 cells are seen as one-third of a 117-cell structure that was already complete. No value was added. No seed was imported. No fourth level was built. The only thing that changed was the ruleset — not by a new instruction, but by a new reading of the instructions already present.

The tensor is still the operand. The zones are what the operand looks like when its own operation is turned on it.

The construction did not take a tenth step. It noticed that the ninth step had completed more than it appeared to.

5. The next pass

Fractal rule. The same four rules, the same triadic rhythm, the same three-zone pattern, applied to the completed hyper-structure as a new operand at the next scale.

Nothing is added. There is still no ending clause, so the 117-point object is not a terminus but a compacted operand.

Our hyper-tensor has officially become a “point” again. For clarity, this lets us treat it once again as a structurally 1-dimensional construction.

Two counts. 117 counts lattice points. And structural "dimension" counts levels of structural nesting: a tensor becomes a hyper-tensor, then a lattice of hyper-tensors, and so on. The first is a count of values; the second is a count of address structures. They must be kept apart.

The register. The first dimensional count gives the hyper-structure as 4-dimensional (a hyper-tensor). The Bitriadic sequence expands it by six more, totalling 10, and then a carry resets the count:

Register	Structure	Instance
D1–D3	Tensor structure (M-meta)	Tensor
D4	Hyper-tensor (H-meta)	Hyper-tensor
D5	Lattice of hyper-tensors	**Point’**
D6	Hyper-lattice of hyper-tensors	String’
D7	Lattice of hyper-lattices	Matrix’
D8	Hyper-lattice of hyper-lattices	Tensor’
D9	Hyper-hyper-lattice of hyper-lattices	Hyper-tensor’
D10	Hyper-hyper-hyper-lattice of hyper-hyper-lattices	**?**

The right-hand column is the tell. At D5 the register restarts at point — the lowest instance — even though what it is naming is a lattice of hyper-tensors. It then climbs the same rungs as §1–§3: string, matrix, tensor, hyper-tensor. By D10 it has reached

hyper-tensor again and has no name for what comes next. The ladder has run its course a second time, and the same limit that blocked the fourth advance at step 9 blocks D11.

This is why the construction ends at 10, and it is not a coincidence. It is the same limit that blocked the fourth advance at step 9, one scale up. Rule 3 permits exactly three instances per cycle; the register permits exactly two cycles. D10 is the second hyper-tensor. D11 would have to name a third reification, and there is no fourth instance to name. The triadic limit is not a feature of one level; it is the shape of the register at every scale. The construction appears to end at dimensional number 10 and to disappear at 11D.

Whether that is a deep coincidence or the edge the grammar was always pointing at is the open question. But you can learn where it came from and where it leads!

THE CLIFFHANGER

The 117 is not the end, it is an attractor in a hyper-dimensional integer fractal.

But what if the seed 2 can actually be a cardinality of a marked pair of operations [+1; −1], and each point in the fractal can be imagined as a distinctive instruction set or a computational agent?

Find out by reading the Poetica Mathematica monograph, which continues the fractal beyond this first construction and provides a unique interpretational lens.

Source boundary: This paper is a standalone reorganization of Interlude I, §§ I.1–I.10 (PDF pp. 58–64) and the corresponding four-rule formulation and level formulas in Chapter I §§ 1.3–1.6 (PDF pp. 37–42). All terms needed for the reading are defined here. The full monograph, *Poetica Mathematica: Cogitatio ex Nihilo et Speculum Scientiae* (Version 0.25), is by Kirill Kovlekov (2026) and is available via Zenodo at https://doi.org/10.5281/zenodo.23698076.

FAQ

So what did I just build?

A generative integer fractal. One seed, four rules, and a completed ten-step pass. The 117-point hyper-structure is the first completed pass, not the end of the generative schema. The same four rules can be applied again at a larger scale. What repeats is not a picture; the grammar does.

Isn't this cheating?

If a construction may invent a rule whenever it gets stuck, it can reach any destination it likes. So the question is whether the turns are forced or chosen.

They are forced by ***structural least-action***: at each stage the structure takes the smallest change that lets it continue, importing no new seed, foreign operation, or outside object. When the fourth advance is blocked, the next move is read off the pattern already present: a first form, a doubled form, a tripled form. The zone names are labels for those derived forms, not imported ingredients.

This does not mean the system is confined to a narrow menu of pre-approved moves. The rule-set sets the boundary, not the itinerary. Anything the system can build inside that boundary is fair game, however much internal complexity it generates. The constraint is that nothing crosses the boundary: no external entity, no foreign operation, no new seed. Internal elaboration is unconstrained.

The turns look optimized, but no foresight is involved. The system runs inside the rules it has, tries what those rules allow, and fails where they run out. Each failure remains in the final structure as a compressed record of a move that was made and could not be carried forward. Those records accumulate until the current rule-set is saturated and no legal move remains. That saturation, not convenience, forces the turn into a new structural level or operation.

So structural least-action is a retrospective description, not a plan. The finished object shows only the working path; the turns are the visible trace of what the system could not do. It looks like minimal design because it is what survives after everything that did not work has been ruled out - not because anything was designed.

Is it really a fractal?

Yes, but the self-similarity is in the rules, not in the picture. Most fractals repeat a shape inside a shape; this one repeats a procedure inside a procedure. At every scale the structure is built the same way: a seed, a triadic advance, a blocked fourth step, a forced sideways turn, three zones. The values grow enormous and the addresses change rank, but the four rules remain the same. That is the self-similarity.

Why not write down the re-seeding map?

Because it cannot be written as a single finite formula, and that is a property of the object rather than a gap in the exposition. The map that carries one completed structure into the next seed is a schema, not a closed-form function. At every finite stage it is fully determined by the four rules: read the completed hyper-structure as a new seed, run the same triadic advance, let the blocking rule fire, take the sideways turn. What cannot be done is collapse that schema into one finished expression, because the construction is transfinite. There is no final stage at which the map could be exhibited as a completed object.

For a specialist, the precise form of the claim is: the re-seeding map is well defined at every successor stage, and limit stages are taken as the union of everything built so far. State that and the objection turns into a description of the object.

What stays the same across scales?

The rule-set. That is the whole invariant. The cell values, the index arity, the operation rank, and the coordinate axes all change between passes. The four rules do not. Neither does the three-zone pattern - prime, reflection, projection - nor the triadic rhythm of three positions before the form changes. Everything else is scale-dependent; those are not.

FAQ · continued

What plays the role of dimension?

Three notions must be kept distinct. A structural step is a move in the ten-step construction. Index arity is the number of indices in the hindsight address: one for the point-values, two for the matrix pattern, and three for the tensor pattern. The structural degree-of-freedom count adds the zone coordinate introduced at the grammatical step.

The count is a schedule rather than a constant. Before the recognition, index arity rises from one to two to three. After the grammatical turn, the extra zone coordinate is active: the L, M, and H levels have two, three, and four address coordinates respectively. This is not Hausdorff dimension or a measurement of how a set fills physical space; it is a count of the structural coordinates used to locate a cell.

Isn't this just an L-system with extra labels?

No, and the difference is the point. An L-system rewrites symbols in parallel over a fixed alphabet with a fixed rule; its algebra never changes rank. Here two things escalate together. The index arity rises - one index, then two, then three - and the operation rank rises with it - addition at the first level, multiplication at the second, exponentiation at the third. Coupled escalation is the first difference.

The second is the blocking rule. In an L-system, the rewriting rule is the generator and nothing blocks. Here the failure of the fourth repetition is the generator: the ban on a fourth repetition is what forces the sideways turn, and the sideways turn is what produces the next scale. The fractal is driven by a prohibition, not by a production.

And it is integer throughout. Every cell is an exact whole number - powers, self-powers, power towers - with no approximations and no continuous limit anywhere. A discrete fractal with no metric limit is unusual enough to be worth naming.

Why call it a lattice?

"Lattice" here means an indexed array of cells - first points, then strings, then matrices, then the hyper-structure - not an order-theoretic lattice and not a discrete subgroup of a vector space. The word is doing descriptive work, not technical work.